Assignment 1
Set A

Course: B. Tech. Year/Semester: III & V **Session:** 2026-2027

Subject Name & Code: Design & Analysis of Algorithms (BCSC1012)

1. Time Complexity of Code Snippets

Determine the time complexity of the following code snippet using Big-O notation. Also justify your answer step by step.

```
for(i = 1; i <= n; i++) { for(j
  = 1; j <= i; j++) {
    count++;
  }
}
```

2. Asymptotic Notations

For the function **$f(n) = 5n^2 + 3n + 7$**, find suitable values of **c_1, c_2, and n_0** to prove that **$f(n) = \Theta(n^2)$** using the formal definitions of asymptotic notation.

3. Recurrence Relations — Master Theorem & Substitution Method

Solve the recurrence relation **$T(n) = 2T(n/2) + n$** using both the **Master Theorem** and the **Substitution Method**. Show the major steps and give the final asymptotic complexity.

4. Quick Sort

Apply **Quick Sort** to the array **[35, 12, 43, 8, 25, 17, 50]**, taking the last element as the pivot. Show each partitioning step and state the best-case, average-case, and worst-case time complexities.

5. Merge Sort

Apply **Merge Sort** to the array **[38, 27, 43, 3, 9, 82, 10]**. Show the complete divide-and-merge process and derive the time complexity using its recurrence relation.

6. Heap Sort

Construct a **Max Heap** from the array **[20, 15, 30, 5, 10, 25, 35]**. Then use Heap Sort to arrange the elements in ascending order. Show the major heapify steps and state the time complexity.

7. Insertion Sort

Perform **Insertion Sort** on the array **[29, 10, 14, 37, 13]**, showing the array after every pass. Also state its best-case and worst-case time complexities and explain when Insertion Sort is preferable.

8. Counting Sort

Use **Counting Sort** to sort the array **[4, 2, 2, 8, 3, 3, 1]**. Show the count/frequency array, cumulative count if used, and the final sorted array. State the time and space complexities.

9. Red-Black Tree

Insert the keys **10, 20, 30, 15, 25** into an initially empty **Red-Black Tree**. Draw the tree after the required rotations and recoloring operations, and explain which Red-Black Tree properties are restored.

10. Binomial Heap

Insert the keys **10, 20, 5, 30, 15, and 8** into a **Binomial Heap**. Show how the binomial trees are linked after each necessary union and state the time complexity of the main operations.

11. B-Tree

Construct a **B-Tree of minimum degree 2** by inserting the keys **10, 20, 5, 6, 12, 30, 7, and 17**. Show all node-splitting operations and draw the final B-Tree.

12. What is the complexity of the following algorithm?

```
for (i = 0; i< n; i ++)
        for (j = 0; j < n; j ++)
                a[i][j] = b[i][j] * x;
```

13. Write an algorithm of Shell Sort and compute the running time complexity. Also compute the complexity for increasing order sorted data, decreasing order sorted data and random data.

14. Apply the Heap sort [Min] on the given input Array [11, 13, 18, 6, 9, 14, 18, 20, 19, 5].

15. Suppose we want to search for a number in a sorted list of numbers. Which searching algorithm is best suited for this task? Write a recurrence equation for this algorithm. Solve the recurrence equation.

Assignment 1
Set B

Course: B. Tech. Year/Semester: III & V **Session:** 2026-2027

Subject Name & Code: Design & Analysis of Algorithms (BCSC1012)

1. Time Complexity of Code Snippets

Analyze the time complexity of the following code using Big-O notation. Explain the number of iterations performed by the inner loop and derive the final complexity.

```
for (i = 1; i <= n; i = i * 2) {
for (j = 1; j <= n; j++) {
count++;
}
}
```

2. Asymptotic Notations

Using the formal definitions of Big-O, Ω, and Θ, prove that $f(n) = 8n^2 + 5n + 12$ is $\Theta(n^2)$. Determine suitable values of c_1, c_2, and n_0.

3. Recurrence Relations — Master Theorem & Substitution Method

Solve the recurrence relation $T(n) = 4T(n/2) + n$ using the Master Theorem. Then verify the result using the substitution method and state the final asymptotic complexity.

4. Quick Sort

Apply Quick Sort to the array [42, 18, 7, 35, 24, 60, 11], choosing the first element as the pivot. Show the partitioning process at each stage and discuss the best, average, and worst-case time complexities.

5. Merge Sort

Use Merge Sort to sort the array [64, 25, 12, 22, 11, 90, 34]. Show the complete division and merging process. Also write the recurrence relation and determine its time complexity.

6. Heap Sort

Build a Max Heap from the array [16, 14, 10, 8, 7, 9, 3]. Then perform Heap Sort to obtain the elements in ascending order. Show the heap construction and deletion steps, and state the complexity.

7. Shell Sort

Perform Shell Sort on the array [32, 17, 45, 23, 9, 14, 28] using the gap sequence 3, 1. Show the array after each gap pass and explain how Shell Sort improves upon ordinary Insertion Sort.

8. Radix Sort

Apply LSD Radix Sort to the array [329, 457, 657, 839, 436, 720, 355]. Show the sorting result after processing the units, tens, and hundreds digits. State the time and space complexities of Radix Sort.

9. Red-Black Tree

Starting with an empty Red-Black Tree, insert the keys 41, 38, 31, 12, 19, and 8. Draw the tree after each insertion where a rotation or recoloring is required, and explain the fix-up cases used.

10. Binomial Heap

Create a Binomial Heap by inserting the elements 12, 7, 25, 15, 28, and 10. Show the binomial trees after consolidation and explain the Union and Extract-Min operations along with their time complexities.

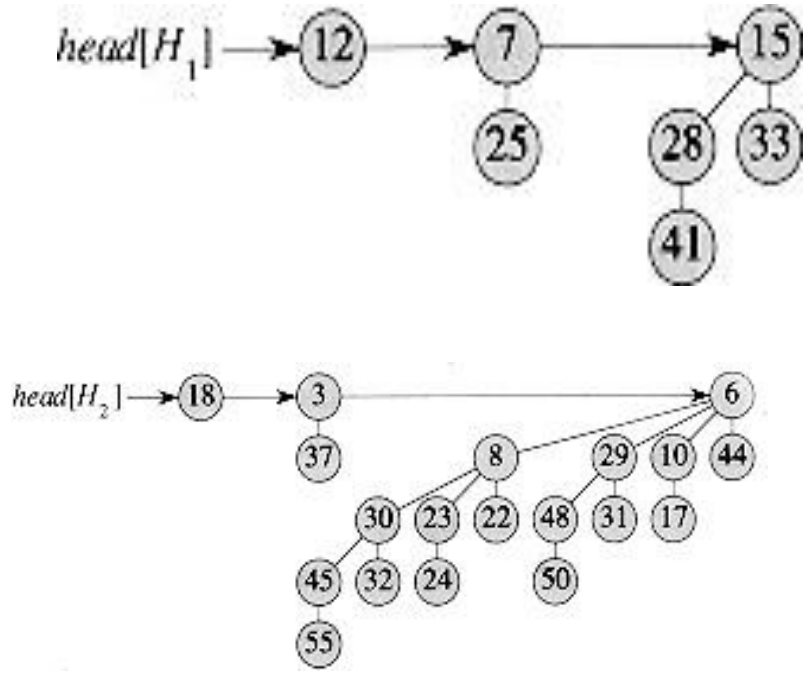
11. B-Tree

Construct a B-Tree of minimum degree 3 by inserting the keys 10, 20, 5, 6, 12, 30, 7, 17, 25, and 40. Show the node-splitting operations and draw the final B-Tree.

12. What do you understand by Divide-Conquer Approach? Write Quick Sort algorithm and apply it to the given data 12, 32, 9, 5, 2, 4. Also analyze the complexity if data is already sorted.

13. Execute the Merge sort on the given input Array [11, 23, 18, 7, 35, 14, 17, 20,27, 16].

14. Discuss about the Binomial Heap and its properties. Create a binomial heap using given (a) and (b) binomial heaps.



Assignment 1
Set C

Course: B. Tech. Year/Semester: III & V **Session:** 2026-2027

Subject Name & Code: Design & Analysis of Algorithms (BCSC1012)

1. Time Complexity of Code Snippets

Analyze the time complexity of the following code using Big-O notation. Explain the number of iterations performed by the outer and inner loops and derive the final complexity.

```
for(i=1;i<=n;i=i*2){
   for(j=1;j<=i;j++){
      count++;
   }}
```

2. Asymptotic Notations

Using the formal definitions of Big-O, Ω, and Θ, prove that $f(n) = 6n^2 + 4n + 10$ is $\Theta(n^2)$. Determine suitable values of c_1, c_2, and n_0.

3. Recurrence Relations — Master Theorem & Substitution Method

Solve the recurrence relation $T(n) = 2T(n/2) + n$ using the Master Theorem. Then verify the result using the substitution method and state the final asymptotic complexity.

4. Quick Sort

Apply Quick Sort to the array [29, 10, 14, 37, 13, 25, 8], choosing the last element as the pivot. Show the partitioning process at each stage and discuss the best, average, and worst-case time complexities.

5. Merge Sort

Use Merge Sort to sort the array [45, 12, 33, 7, 18, 2, 50]. Show the complete division and merging process. Also write the recurrence relation and determine its time complexity.

6. Heap Sort

Build a Max Heap from the array [40, 15, 30, 10, 25, 5, 35]. Then perform Heap Sort to obtain the elements in ascending order. Show the heap construction and deletion steps, and state the complexity.

7. Shell Sort

Perform Shell Sort on the array [19, 2, 31, 45, 6, 11, 121, 27] using the gap sequence 4, 2, 1. Show the array after each gap pass and explain how Shell Sort improves upon ordinary Insertion Sort.

8. Counting Sort

Apply Counting Sort to the array [3, 6, 4, 1, 3, 4, 1, 2]. Show the frequency/count array and the final sorted result. State the time and space complexities of Counting Sort.

9. Red-Black Tree

Starting with an empty Red-Black Tree, insert the keys 20, 15, 25, 10, 5, and 1. Draw the tree after each insertion where a rotation or recoloring is required, and explain the fix-up cases used.

10. Binomial Heap

Create a Binomial Heap by inserting the elements 12, 7, 25, 15, 28, 10, and 20. Show the binomial trees after consolidation and explain the Union and Extract-Min operations along with their time

complexities.

11. B-Tree

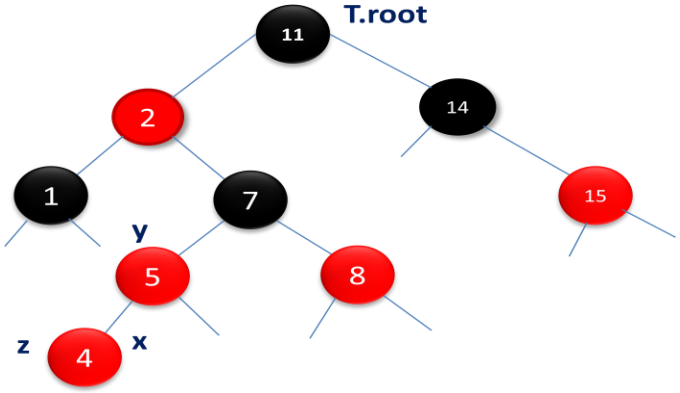
Construct a B-Tree of minimum degree 2 by inserting the keys 15, 10, 20, 8, 12, 17, 25, and 30. Show the node-splitting operations and draw the final B-Tree.

12. Insert the following numbers in B-Tree of minimum degree 2.

34,22,32,11,21,14,16,2,4,7,1,18,44,33,65,78,76,29,50,40,30,45

13. Construct the binomial heap for the following sequence of numbers7,2,4,17,1,11,6,8,15,10,20. Also apply the operation of extracting the minimum key from the binomial heap.

14. An element z has been inserted into a Red-Black Tree in which 2, 4, 5, 8 and 15 are red nodes and others are black. After insertion, the rules of Red-Black Tree is violated, perform the suitable operation to satisfy all the properties of Red-Black Tree.



Assignment 1
Set D

Course: B. Tech. Year/Semester: III & V **Session:** 2026-2027

Subject Name & Code: Design & Analysis of Algorithms (BCSC1012)

1. Time Complexity of Code Snippets

Analyze the time complexity of the following code using Big-O notation. Explain the number of iterations performed by each loop and derive the final complexity.

```
for (i = 1; i <= n; i++) {
   for (j = 1; j <= i; j++) {
      count++;
   }
}
```

2. Asymptotic Notations

Using the formal definitions of Big-O, Ω, and Θ, prove that $f(n) = 5n^3 + 2n^2 + 10$ is $\Theta(n^3)$. Determine suitable values of c_1, c_2, and n_0.

3. Recurrence Relations — Master Theorem & Substitution Method

Solve the recurrence relation $T(n) = 3T(n/2) + n$ using the Master Theorem. Then verify the result using the substitution method and state the final asymptotic complexity.

4. Quick Sort

Apply Quick Sort to the array [50, 23, 9, 18, 61, 32, 7], choosing the first element as the pivot. Show the partitioning process at each stage and discuss the best, average, and worst-case time complexities.

5. Merge Sort

Use Merge Sort to sort the array [64, 34, 25, 12, 22, 11, 90]. Show the complete division and merging process. Also write the recurrence relation and determine its time complexity.

6. Heap Sort

Build a Max Heap from the array [18, 40, 12, 25, 30, 8, 50]. Then perform Heap Sort to obtain the elements in ascending order. Show the heap construction and deletion steps, and state the complexity.

7. Shell Sort

Perform Shell Sort on the array [20, 35, 15, 7, 55, 1, 22] using the gap sequence 4, 2, 1. Show the array after each gap pass and explain how Shell Sort improves upon ordinary Insertion Sort.

8. Radix Sort

Apply LSD Radix Sort to the array [170, 45, 75, 90, 802, 24, 2, 66]. Show the sorting result after processing the units, tens, and hundreds digits. State the time and space complexities of Radix Sort.

9. Red-Black Tree

Starting with an empty Red-Black Tree, insert the keys 41, 38, 31, 12, 19, and 8. Draw the tree after each insertion where a rotation or recoloring is required, and explain the fix-up cases used.

10. Binomial Heap

Create a Binomial Heap by inserting the elements 10, 20, 30, 40, 50, and 60. Show the binomial trees after consolidation and explain the Union and Extract-Min operations along with their time complexities.

11. B-Tree

Construct a B-Tree of minimum degree 2 by inserting the keys 8, 9, 10, 11, 15, 20, 17, and 16. Show the node-splitting operations and draw the final B-Tree.

12. There is a huge amount of data of a university; a sorting algorithm is required to apply over the data to sort according to the student's roll number. Which sorting algorithm you will apply to arrange the data in ascending order? Also derive the complexity in best, worst and average case complexity.

13. Compute the complexity of given recurrence: $T(n) = 2T(\sqrt{n}) + 1$

14. Show that for any real constant a and b, where

$b > 0$, $(n+a)^b = \theta(n^b)$

15. Find Θ bound for $f(n)=n^2/2 - n/2$.

Assignment 1
Set E

Course: B. Tech. Year/Semester: III & V **Session:** 2026-2027

Subject Name & Code: Design & Analysis of Algorithms (BCSC1012)

1. Time Complexity of Code Snippets

Analyze the time complexity of the following code using Big-O notation. Explain the number of iterations performed by the nested loops and derive the final complexity.

```
i = 1;
while (i < n) {
   j = 1;
   while (j < n) {
      j = j * 2;
   }
   i = i * 2;
}
```

2. Asymptotic Notations

Using the formal definitions of Big-O, Ω, and Θ, prove that $f(n) = 7n^2 + 3n + 9$ is $\Theta(n^2)$. Determine suitable values of c_1, c_2, and n_0.

3. Recurrence Relations — Master Theorem & Substitution Method

Solve the recurrence relation $T(n) = T(n/2) + 1$ using the substitution method. Then verify the result using the Master Theorem and state the final asymptotic complexity.

4. Quick Sort

Apply Quick Sort to the array [31, 14, 59, 26, 41, 58, 9], choosing the last element as the pivot. Show the partitioning process at each stage and discuss the best, average, and worst-case time complexities.

5. Merge Sort

Use Merge Sort to sort the array [21, 10, 45, 3, 18, 7, 30, 12]. Show the complete division and merging process. Also write the recurrence relation and determine its time complexity.

6. Heap Sort

Build a Max Heap from the array [22, 13, 17, 11, 10, 14, 16]. Then insert the element 25 into the heap and perform Heap Sort. Show the heap construction, insertion, and deletion steps, and state the complexity.

7. Shell Sort

Perform Shell Sort on the array [9, 8, 3, 7, 5, 6, 4, 1] using the gap sequence 4, 2, 1. Show the array after each gap pass and explain how Shell Sort improves upon ordinary Insertion Sort.

8. Counting Sort

Apply Counting Sort to the array [7, 2, 9, 4, 2, 7, 1, 5]. Show the count array, cumulative count array, and final sorted result. State the time and space complexities of Counting Sort.

9. Red-Black Tree

Starting with an empty Red-Black Tree, insert the keys 30, 15, 60, 10, 20, 40, and 70. Draw the tree after each insertion where a rotation or recoloring is required, and explain the fix-up cases used.

10. Binomial Heap

Create a Binomial Heap by inserting the elements 18, 8, 25, 5, 12, 20, and 30. Show the binomial trees after consolidation and explain the Union and Extract-Min operations along with their time complexities.

11. B-Tree

Construct a B-Tree of minimum degree 2 by inserting the keys 5, 10, 15, 20, 25, 30, 35, and 40. Show the node-splitting operations and draw the final B-Tree.

12. Banks often record transactions on an account in order of the times of the transactions, but many people like to receive their bank statements with checks listed in order by check number. People usually write checks in order by check number, and merchants usually cash them with reasonable dispatch The problem of converting time-of-transaction ordering to check-number ordering is therefore the problem of sorting almost sorted input. Argue that the procedure Insertion-sort would tend to beat the procedure QuickSort on this problem.

13. Write the program with the help of Quick sort on the given input Array [35, 26, 18, 9, 35,14, 29, 20, 36, 19]. Pivot element is 35.

14. Discuss the stable property. Perform the sorting operation and show the stable property on given data: A={5,0,5,0,1,7,4,7,1,3,2}.

15. Write the algorithm for counting sort. Illustrate the operation of counting sort on an array A={6,0,2,0,1,3,4,6,1,3,2}.